

Nouvelle fonctionnalité : Ajout d'image pour les piscines

Objectif

Ajouter la possibilité d'associer une image à chaque piscine dans l'application. Les utilisateurs pourront téléverser une image lors de l'ajout ou de la modification d'une piscine, et celle-ci sera affichée dans la liste des piscines.

Contexte

Actuellement, l'application permet de gérer des piscines (ajout, modification, suppression, affichage) mais ne gère pas d'images. Vous allez implémenter cette fonctionnalité de bout en bout.

Grandes étapes

1. Préparer la base de données

- ☐ Ajouter une colonne **image** (VARCHAR 255) dans la table **piscine** de votre base de données
- ☐ Cette colonne contiendra le chemin relatif vers l'image stockée sur le serveur

Exemple SQL :

```
ALTER TABLE piscine ADD COLUMN image VARCHAR(255) NULL;
```

2. Modifier la classe **Piscine**

- ☐ Ajouter une propriété privée **#image** dans le constructeur
- ☐ Ajouter le paramètre **image** dans le constructeur et l'assigner à la propriété
- ☐ Créer un getter **get image()** pour accéder à cette propriété
- ☐ Modifier la méthode **injecterHTML()** pour afficher l'image dans le HTML
 - ☐ Ajouter une balise **** avec l'attribut **src** pointant vers l'image
 - ☐ Gérer le cas où aucune image n'est disponible (image par défaut ou placeholder)

Fichier concerné : **Piscine.js**

3. Modifier le formulaire d'ajout/modification

3.1 Modifications HTML

- ☐ Dans le fichier HTML du formulaire, ajouter un champ **<input type="file">** avec les attributs :
 - ☐ **accept="image/*"** pour n'accepter que les images
 - ☐ Un attribut **data-*** pour le sélectionner facilement (ex: **data-image**)
 - ☐ Un **id** et un **name** appropriés

Exemple :

```
<input type="file" id="image" name="image" accept="image/*" data-image />
```

3.2 Modifications JavaScript

- ☐ Dans la classe `Formulaire`, récupérer la référence à ce champ dans le constructeur
- ☐ Lorsque la première requête de soumission du formulaire est faite, récupérer le fichier sélectionné et enchaîner une deuxième promesse pour le téléversement de l'image
- ☐ Modifier la méthode de soumission du formulaire pour :
 - ☐ Utiliser `FormData` au lieu de `JSON.stringify()` pour le body de la requête pour l'ajout/modification de l'image
 - ☐ Ajouter le fichier image au FormData avec `formData.append('image', fichier)`

Exemple de code pour récupérer le fichier :

```
// Dans la classe Formulaire

// Récupérer la référence au champ file dans le constructeur
this.#champImageHTML = this.#conteneurHTML.querySelector("[data-image]");

// Dans la méthode de soumission du formulaire
#onSubmitFormulaire(event) {
    event.preventDefault();

    // Récupérer le fichier sélectionné
    if (this.#champImageHTML.files.length === 0) {
        console.log("Aucun fichier sélectionné.");
        return;
    }
    const fichierImage = this.#champImageHTML.files[0];

    // Vérifier qu'un fichier a été sélectionné
    if (fichierImage) {
        console.log("Nom du fichier:", fichierImage.name);
        console.log("Type du fichier:", fichierImage.type);
        console.log("Taille du fichier:", fichierImage.size, "octets");

        // Créer un FormData pour envoyer le fichier
        const formData = new FormData();
        formData.append('image', fichierImage);
        formData.append('id', donneesPiscine.id); // Si modification

        // Envoyer le fichier à l'API
        this.#application.ajouterImage(formData);
    }
}
```

Dans la méthode `ajouterImage()` de la classe `Application`, modifier la requête fetch pour utiliser `formData` comme body.

```
// Dans la classe Formulaire
#onSubmitFormulaire(evenement) {
    evenement.preventDefault(); // On bloque l'envoi de formulaire par défaut

    if (this.#validerFormulaire()) {
        //Envoyer les infos de notre formulaire
        this.#champsHTML.forEach(
            function (champ) {
                const name = champ.name;

                // Valider le fichier avant de l'envoyer
                if (champ.name === "image" ) {
                    const fichier = champ.files[0];

                    console.log("Nom du fichier:", fichier.name);
                    console.log("Type du fichier:", fichier.type);
                    console.log("Taille du fichier:", fichier.size, "octets");

                    // Vérifier qu'un fichier a été sélectionné
                    if (!fichier || champ.files.length === 0) {
                        new ToastErreur(document.body, "Aucun fichier
sélectionné.");

                        return;
                    }

                    // Validation du type
                    const typesAcceptes = ["image/jpeg", "image/png",
"image/gif", "image/webp"];
                    if (!typesAcceptes.includes(fichier.type)) {
                        new ToastErreur(document.body, "Format d'image non
accepté. Utilisez JPG, PNG, GIF ou WebP.");
                        return;
                    }

                    // Validation de la taille (5 MB = 5 * 1024 * 1024 octets)
                    const tailleMax = 5 * 1024 * 1024;
                    if (fichier.size > tailleMax) {
                        new ToastErreur(document.body, "L'image est trop
volumineuse. Taille maximale: 5 MB.");
                        return;
                    }

                    this.#fichierImage = fichier; // On peut stocker le nom du
fichier ou son chemin
                }else {
                    const value = champ.value;
                    this.#donneesFormulaire[name] = value;
                }
            }.bind(this)
        );
    }
}
```

```

    );

    if (this.#methode === "POST") {
        this.#application.ajouterPiscine(this.#donneesFormulaire,
this.#fichierImage);
    } else if (this.#methode === "PUT") {
        this.#application.modifierPiscine(this.#donneesFormulaire,
this.#fichierImage);
    }
}
}
}

```

```

// Dans la classe Application
ajouterPiscine(nouvellePiscine, fichierImage) {
    const config = {
        method: "POST",
        header: {
            "Content-Type": "application/JSON",
        },
        body: JSON.stringify(nouvellePiscine),
    };

    fetch(`http://localhost:8888/api/piscine/ajouterUn.php`, config)
        .then(
            function (reponse) {
                return reponse.json();
            }.bind(this)
        )
        .then(
            function (donnees) {
                const formData = new FormData();
                formData.append('image', fichierImage);
                formData.append('id', donnees.id); // Nécessaire pour lier
l'image à la piscine

                const config = {
                    method: "POST",
                    body: formData, // Utilisation de FormData
                };

                return
fetch(`http://localhost:8888/api/piscine/ajouterImage.php`, config);
            }.bind(this)
        )
        .then(
            function (reponse) {
                return reponse.json();
            }.bind(this)
        )
        .then(
            function (donnees) {
                this.#formulaire.viderFormulaire();
            }
        )
    }
}

```

```

        this.changerPage(this.#pagination.pageCourante,
this.#pagination.itemsParPage);
    }.bind(this)
    );
}

```

Fichier concerné : `Formulaire.js` et `Application.js`

4. Créer le script PHP pour gérer le téléversement

- ☐ Créer un nouveau fichier `ajouterImage.php` dans le dossier API
- ☐ Implémenter la logique de téléversement :
 - ☐ Valider le type de fichier (JPEG, PNG, GIF, WEBP uniquement)
 - ☐ Valider la taille du fichier (max 5 MB)
 - ☐ Recevoir les données en `FormData` plutôt qu'en JSON
 - ☐ Récupérer le fichier depuis `$_FILES['image']`
 - ☐ Générer un nom unique pour éviter les conflits (ex: `uniqid() . '_' . nom_original`)
 - ☐ Créer le dossier `assets/img` s'il n'existe pas
 - ☐ Déplacer le fichier téléversé dans `assets/img/`
 - ☐ Enregistrer le chemin de l'image dans la base de données dans la colonne `image` de la piscine correspondante
 - ☐ Retourner le chemin relatif du fichier en JSON
 - ☐ Gérer les erreurs de téléversement avec des messages appropriés

Types MIME acceptés :

- `image/jpeg`
- `image/png`
- `image/gif`
- `image/webp`

Fichier à créer : `api/piscine/ajouterImage.php`

6. Modifier la méthode `recupererDonnees()`

- ☐ Dans `Application.js`, s'assurer que l'API `rechercherPagination.php` retourne le champ `image`
- ☐ Passer ce paramètre au constructeur de `Piscine` lors de la création des instances
- ☐ Vérifier que la destructuration inclut `image`

Fichier concerné : `Application.js`

8. Modifier la méthode `changerPage()`

- ☐ S'assurer que les données paginées incluent le champ `image`
- ☐ Passer ce paramètre au constructeur de `Piscine` lors de la création des instances
- ☐ Vérifier que la destructuration inclut `image`

9. Gérer la suppression d'image

- ☐ Dans `supprimerUn.php`, ajouter la logique pour :
 - ☐ Récupérer le chemin de l'image avant de supprimer l'enregistrement
 - ☐ Vérifier si le fichier existe sur le serveur avec `file_exists()`
 - ☐ Supprimer le fichier image du serveur avec `unlink()`
 - ☐ Supprimer l'enregistrement de la base de données
 - ☐ Gérer les erreurs si la suppression du fichier échoue

Fichier concerné : `api/piscine/supprimerUn.php`

```
// Dans supprimerUn.php
//... Récupération de l'ID à supprimer via $_GET
$id = $_GET['id'] ?? null;

// Récupérer le chemin de l'image avant de supprimer l'enregistrement
$requeteSelect = $connexion->prepare("SELECT image FROM piscine WHERE id = :id");
$requeteSelect->bindParam(':id', $id, PDO::PARAM_INT);
$requeteSelect->execute();

$piscine = $requeteSelect->fetch(PDO::FETCH_ASSOC);

if (!$piscine) {
    header('Content-Type: application/json');
    http_response_code(404); // Not Found
    $resultat = ["success" => false, "message" => "Aucun enregistrement trouvé avec cet ID"];
    echo json_encode($resultat);
    exit;
}

$cheminImage = $piscine['image'];

// Supprimer l'image sur le serveur si l'information est disponible
if (!empty($cheminImage)) {
    $cheminComplet = __DIR__ . '/../..' . $cheminImage;

    // Vérifier si le fichier existe sur le serveur
    if (file_exists($cheminComplet)) {
        // Tenter de supprimer le fichier
        if (!unlink($cheminComplet)) {
            // Gérer l'erreur si la suppression échoue
            header('Content-Type: application/json');
            http_response_code(404); // Not Found
            $resultat = ["success" => false, "message" => "Impossible de supprimer le fichier image : " . $cheminImage];
            echo json_encode($resultat);
            exit;
        }
    }
}
```

```
    } else {  
        // Le fichier n'existe pas, on peut logger cette information si  
nécessaire  
        error_log("Le fichier image à supprimer n'existe pas : " .  
$cheminComplet);  
    }  
}  
//... Suite de la suppression de l'enregistrement dans la base de données
```

10. Améliorer l'expérience utilisateur (Bonus)

- ☐ Ajouter une prévisualisation de l'image avant l'envoi du formulaire
 - ☐ Écouter l'événement `change` sur le champ file
 - ☐ Utiliser `FileReader` pour lire le fichier
 - ☐ Afficher l'aperçu dans une balise `<img>`
- ☐ Ajouter un bouton pour supprimer/changer l'image lors de la modification
- ☐ Ajouter des messages d'erreur clairs en cas de problème de téléversement
 - ☐ Utiliser la classe `ToastErreur` existante
- ☐ Styliser l'affichage des images
 - ☐ Ajouter `border-radius` pour des coins arrondis
 - ☐ Utiliser `object-fit: cover` pour maintenir les proportions

Fichiers concernés : `Formulaire.js`, `Piscine.js`, fichiers CSS

⚠ Points d'attention

Sécurité

- ☒ Toujours valider le type et la taille des fichiers **côté serveur**
- ☒ Ne jamais faire confiance aux données envoyées par le client
- ☒ Utiliser `mime_content_type()` ou `finfo_file()` pour vérifier le type réel du fichier
- ☒ Générer des noms de fichiers uniques pour éviter les écrasements

Performances

- ☒ Optimiser les images (compression, dimensions appropriées)
- ☒ Limiter la taille maximale (5 MB recommandé)
- ☒ Considérer l'utilisation de formats modernes (WebP)

Organisation

- ☒ Créer une structure de dossiers claire : `uploads/piscines/`. Présentement, les images sont dans `assets/img/` mais il est préférable de les organiser dans un sous-dossier dédié.
- ☒ Ajouter ce dossier au `.gitignore`
- ☒ S'assurer que le serveur a les permissions d'écriture

Gestion d'erreurs

- ☒ Prévoir tous les cas d'erreur possibles

- ☒ Retourner des messages d'erreur explicites
- ☒ Logger les erreurs côté serveur

Accessibilité

- ☒ Ajouter des attributs `alt` descriptifs aux images
- ☒ Utiliser des labels appropriés pour le champ file
- ☒ Indiquer clairement les formats acceptés

🌟 Bonus (optionnel)

- ☐ Implémenter un système de recadrage d'image côté client (ex: avec [Cropper.js](#))
- ☐ Générer automatiquement des miniatures (thumbnails) avec différentes tailles
- ☐ Permettre plusieurs images par piscine (galerie d'images)
- ☐ Ajouter un effet de zoom au survol de l'image (lightbox)
- ☐ Implémenter le drag & drop pour le téléversement
- ☐ Ajouter une barre de progression lors du téléversement
- ☐ Compresser automatiquement les images côté client avant l'envoi

📄 Ressources utiles

- [MDN - FormData](#)
 - [MDN - FileReader API](#)
 - [PHP - Gestion des fichiers téléversés](#)
 - [PHP - move_uploaded_file\(\)](#)
-